
Language Classification using N-grams Accelerated by FPGA-based Bloom Filters

Arpith Jacob & Maya Gokhale Washington University in St. Louis | Lawrence Livermore National Laboratory



The Problem: Language Classification at Scale



What is Language Classification?

Classifying text documents by language — critical for search engines, spam filtering, information retrieval, and text mining.



The Scalability Challenge

Document growth on the web has surpassed Moore's Law improvements. General-purpose CPUs cannot keep up with the data-intensive classification demands.



FPGA as the Solution

FPGAs offer highly parallel architectures to exploit data parallelism — enabling hardware-accelerated classification far beyond software speeds.

Key Results

85x

faster than software
(Mguesser on AMD Opteron)

1.45x

faster than HAIL
(prior FPGA implementation)

99.45%

classification accuracy
(10 European languages)

470

MB/sec throughput
(on XtremeData XD1000)

The Problem

Identifying the language of a document — fast, accurate, and at web scale

What language is this?

“La vie est belle quand on partage...”



FRENCH

“Die Sprache ist der Schlüssel zur Welt...”



GERMAN

“El tiempo es oro, no lo desperdices...”



SPANISH

Why it matters



Search engines

Route queries to correct language-specific indexes



Spam filtering

Flag non-native spam campaigns hitting your inbox



Text mining

Segment multilingual corpora before analysis



Information retrieval

Enable cross-lingual document lookup

BUT...



Documents are growing faster than CPUs

Web-scale document growth far outpaces Moore's-law improvements in general-purpose processors. Software classifiers become the bottleneck — demanding purpose-built hardware acceleration.

Traditional Method

Direct memory lookup tables — store every n-gram, read its language tag

N-GRAM → LANGUAGE TABLE

N-GRAM (key)	LANGUAGE (value)
"the "	EN
"tion"	EN
" der"	DE
"chen"	DE
"cion"	ES
" est"	FR
"tion"	FR

One entry per n-gram · indexed directly by hash address

HOW IT WORKS

1

Build the table offline

For every language, pre-compute its top-t n-grams and write them into a huge memory-mapped table.

2

Hash the n-gram → address

At query time, a document n-gram is hashed to produce an address into the lookup table.

3

Read the stored value

The entry at that address (if any) reveals which language owns this n-gram.

4

Tally & pick the winner

Increment a per-language counter. After streaming the full document, highest count wins.

TRADE-OFF → Simple and exact, but every new language doubles the memory footprint — forcing the table off-chip.

Each Language Has Its Own N-gram Dictionary

A language is represented by its top-t most frequent character n-grams

EN

ENGLISH

“the quick brown fox jumps over...”

TOP N-GRAMS

the·

·the

and·

tion

ing·

ion·

of·t

·of·

that

ther

with

for·

FR

FRENCH

“la vie est belle quand on...”

TOP N-GRAMS

·de·

tion

·le·

ent·

les·

·la·

ion·

ment

que·

ique

·et·

aire

DE

GERMAN

“die Sprache ist der Schlüssel...”

TOP N-GRAMS

·der

der·

die·

und·

·und

chen

ung·

·ein

ein·

icht

·zu·

sche

ES

SPANISH

“el tiempo es oro, no lo...”

TOP N-GRAMS

·de·

de·l

ción

que·

·la·

ació

cion

·que

ent

·el·

los·

para

HOW → Scan a reference corpus for each language, count every 4-character sequence, keep the top 5,000. That set *is* the language's fingerprint.

N-grams: The Core Data Representation

Definition

An n-gram is a contiguous sequence of n characters extracted from a string using a sliding window that shifts one character at a time. For language classification, the paper uses 4-grams ($n = 4$).

Example: "hello" with $n = 4$

h e l l o

"hell"

"ello"

N-gram Profile

1. Extract all 4-grams from a training corpus for each language
2. Rank by frequency — keep the top $t = 5,000$ most common n-grams
3. This profile characterizes the statistical fingerprint of the language

Algorithmic Complexity:

Extracting n-grams from a document of length L produces $L - n + 1$ n-grams. Each must be tested against every language profile — $O(p * (L - n + 1))$ per document for p languages.

N-gram Classification Algorithm



Step 1: Build Profiles

For each of p languages, generate an n -gram profile from a representative training corpus. Keep the top $t = 5,000$ most frequent n -grams.



Step 2: Membership Testing

For an unknown document, extract its n -grams. Test each against every language profile. If present, increment that language's match counter.



Step 3: Classify

The language whose profile has the highest match count is identified as the most probable language of the document.

Key Algorithmic Insight

The bottleneck is membership testing: for each document n -gram, we must probe all p language profiles. Using direct memory lookup tables (like HAIL) requires storing all 5,000 n -grams per language in off-chip SRAM — limiting scalability. The key innovation is replacing these lookup tables with Bloom Filters, a probabilistic data structure that trades a small false positive rate for massive space savings and on-chip parallelism.

Bloom Filters: A Probabilistic Data Structure

A Bloom Filter is a space-efficient probabilistic data structure for set membership testing. It uses k hash functions mapping elements to positions in an m -bit vector. It guarantees no false negatives but may produce false positives.

Bit Vector Visualization ($m = 16$ bits, $k = 3$ hash functions)



False Positive Rate

$$f = (1 - e^{(-N/m)})^k$$

N = n-grams programmed | m = bit-vector length | k = hash functions

Properties

No false negatives

If element is in the set, all k bits will be 1

Possible false positives

Hash collisions may cause bits to be set by other elements

Space-efficient

Order of magnitude smaller than a direct lookup table

Bloom Filters: A Probabilistic Data Structure

Suppose we are building a Bloom filter for the English language profile, and one of the 4-grams in the profile is: *"tion"*

Assume the Bloom filter uses:

- a bit array of length 16
- 3 hash functions

START



Now imagine the hash outputs are:

$h1(\text{"tion"}) = 3$

$h2(\text{"tion"}) = 9$

$h3(\text{"tion"}) = 14$

END



Bloom Filter: SET Operations

SET Operation — Programming a Profile

```
procedure SET(Language profile L)
  BitVector  $\leftarrow$  0
  for each n-gram  $w \in L$ 
    for each hash function  $H_i$ 
      BitVector[ $H_i(w)$ ]  $\leftarrow$  1
    end for
  end for
end procedure
```

Time Complexity: SET is $O(t \cdot k)$ per language | TEST is $O(|D| \cdot k)$ per language | Both operations are highly parallelizable on hardware

Bloom Filter: Step-by-Step Example

Given

Language profile: $L = \{ \text{"tion"}, \text{"ment"} \}$

Bit array size: $m = 10$

Hash functions: $H_1(\text{"tion"})=2, H_2(\text{"tion"})=5, H_3(\text{"tion"})=7$ | $H_1(\text{"ment"})=2, H_2(\text{"ment"})=6, H_3(\text{"ment"})=9$

Step 1: Initialize



Step 2: Insert "tion"



Step 3: Insert "ment"



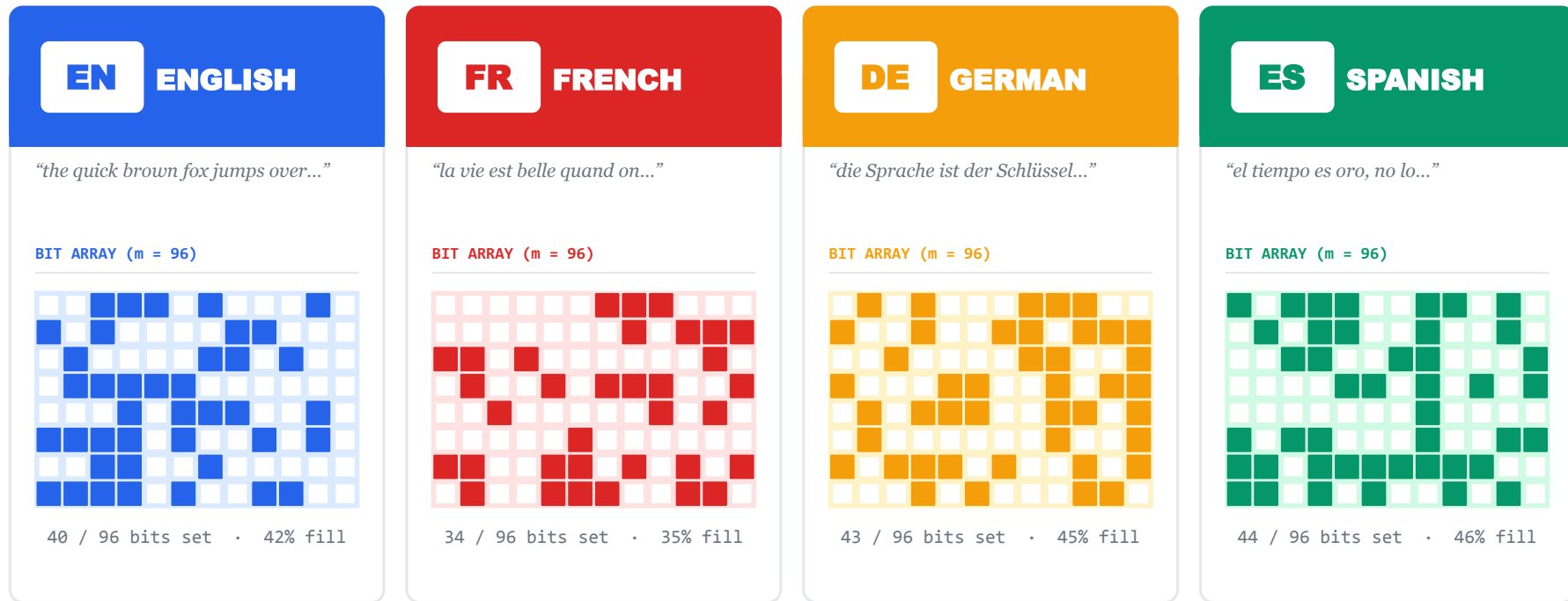
Final BitVector:

[0 0 1 0 0 1 1 1 0 1]

Encodes both "tion" and "ment" — but the original strings cannot be recovered.

Each Language Has Its Own Bit-Array Fingerprint

After hashing, every language's n-gram set collapses into a compact on-chip bit vector



HOW → Every n-gram in a language's profile is hashed by k hash functions and the corresponding bits are set to 1. The resulting pattern is the language's fingerprint — stored entirely on-chip.

Bloom Filter: TEST Operations

TEST Operation — Classifying a Document

```
procedure TEST(Document n-grams D)
  for each n-gram  $w \in D$ 
    Val  $\leftarrow 1$ 
    for each hash function  $H_i$ 
      Val  $\leftarrow$  Val AND BitVector[ $H_i(w)$ ]
    end for
    if Val = 1 then
      match_count++
    end for
  end procedure
```

Time Complexity: SET is $O(t \cdot k)$ per language | TEST is $O(|D| \cdot k)$ per language | Both operations are highly parallelizable on hardware

Bloom Filter TEST: Line-by-Line Example

Bloom Filter BitVector:



```
for each hash  $H_i$ :  
  Val = Val AND BitVector[ $H_i(w)$ ]
```

Case 1: $w = \text{"tion"} \rightarrow \text{MATCH}$

$H_1=2$ $H_2=5$ $H_3=7$

Val = 1 1

Val = 1 AND bit[2]=1 1

Val = 1 AND bit[5]=1 1

Val = 1 AND bit[7]=1 1

Val = 1 $\rightarrow$ match_count++

All k bits were 1 $\rightarrow$ "tion" might be in the set

Case 2: $w = \text{"abcd"} \rightarrow \text{NO MATCH}$

$H_1=2$ $H_2=4$ $H_3=9$

Val = 1 1

Val = 1 AND bit[2]=1 1

Val = 1 AND bit[4]=0 0 $\leftarrow$ one 0 $\rightarrow$ no match

Val = 0 AND bit[9]=1 0

Val = 0 $\rightarrow$ no match

Any 0 bit $\rightarrow$ "abcd" is definitely not in the set

Traditional Lookup Tables

One hash → one memory fetch — but every language needs its own table in software

TIME COMPLEXITY

$$O(N \cdot P)$$

per document, serial software classifier

BREAKDOWN

Per-n-gram lookup (hash + fetch)	$O(1)$
Languages tested per n-gram	P
N-grams in document	N
Total classification time	$O(N \cdot P)$
Space (memory footprint)	$O(P \cdot t)$ • off-chip

Per n-gram: probe each of P language tables sequentially



Bloom Filter Classifier

All P language filters tested in parallel on-chip — collapses a whole factor

TIME COMPLEXITY

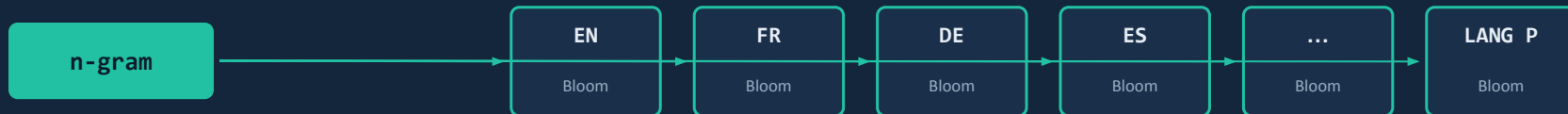
$$O(N \cdot k)$$

per document, with P filters tested in parallel

BREAKDOWN

Per-n-gram test (k hashes + k bit reads)	$O(k)$
Languages tested in parallel	$O(1)$ ← hardware
N-grams in document	N
Total classification time	$O(N \cdot k)$
Space (bit-vectors)	$O(P \cdot m)$ • on-chip

Per n-gram: all P Bloom filters tested simultaneously in one clock



Parallel Bloom Filters: The Key Innovation

Problem with Standard Bloom Filters

Embedded RAMs on FPGAs have a finite number of addressing ports, limiting the number of hash functions that can address the bit-vector simultaneously. A standard Bloom Filter with one shared bit-vector creates a bottleneck.

Solution: Parallel Bloom Filter

Each hash function H_i addresses an independent $1 \times m$ bit-vector, implemented with physically distinct embedded RAMs. This enables all k hash functions to operate in parallel with no port conflicts.

Parallel Bloom Filter Architecture



Uses H_3 family hardware-friendly hash functions (Ramakrishna et al.) for efficient FPGA implementation

Parallel Bloom Filter Classifier

Replicate the classifier p times — process $2p$ n -grams every clock cycle

TIME COMPLEXITY

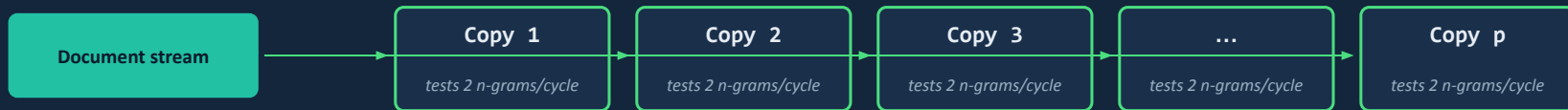
$$O(N/2p)$$

dual-ported RAMs process 2 n -grams/cycle $\times p$ copies

BREAKDOWN

Per- n -gram test (unchanged)	$O(k)$
N -grams per clock cycle	$2p$
N -grams in document	N
Total clock cycles	$O(N / 2p)$
Space (p copies of P filters)	$O(p \cdot P \cdot m)$

Document stream feeds p classifier copies; each copy tests 2 n -grams per clock (dual-port RAMs)



Accuracy vs. Parallelism Trade-off

Bloom Filter parameters directly control the trade-off between classification accuracy and the number of languages supported:

m (Kbits)	k	RAMs / Lang	False Pos. (%)	Accuracy
16	4	64	5	99.45%
8	4	32	44	99.42%
4	6	24	123	99.41%
16	2	32	69	97.31%
8	2	16	209	95.57%

Key Observations

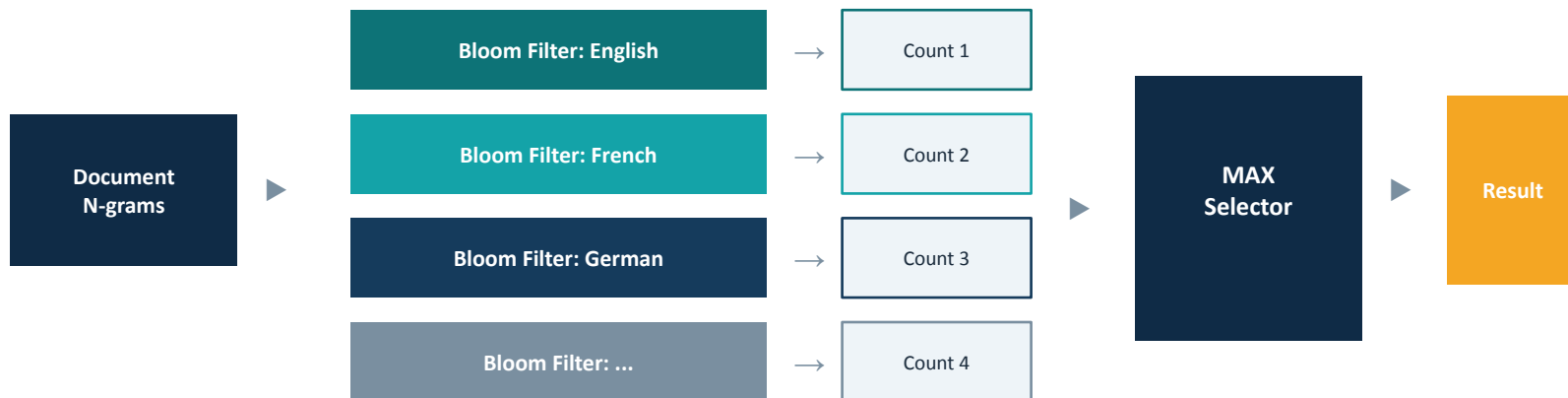
Reducing m from 16 to 4 Kbits (k=6) maintains 99.4% accuracy while using 2.7× fewer RAMs per language

This enables supporting 30 languages vs. only 12 on the same FPGA

Design Space Exploration

With $N = 5,000$ n-grams fixed, the designer can tune k (hash functions) and m (bit-vector size) to balance three competing factors: classification accuracy, embedded RAM usage per language, and total number of languages supported on a given FPGA. The false positive rate is negligible when the match count gap between the top two language candidates is large — which is typically the case for distinct languages.

Multiple Language Classifier Architecture

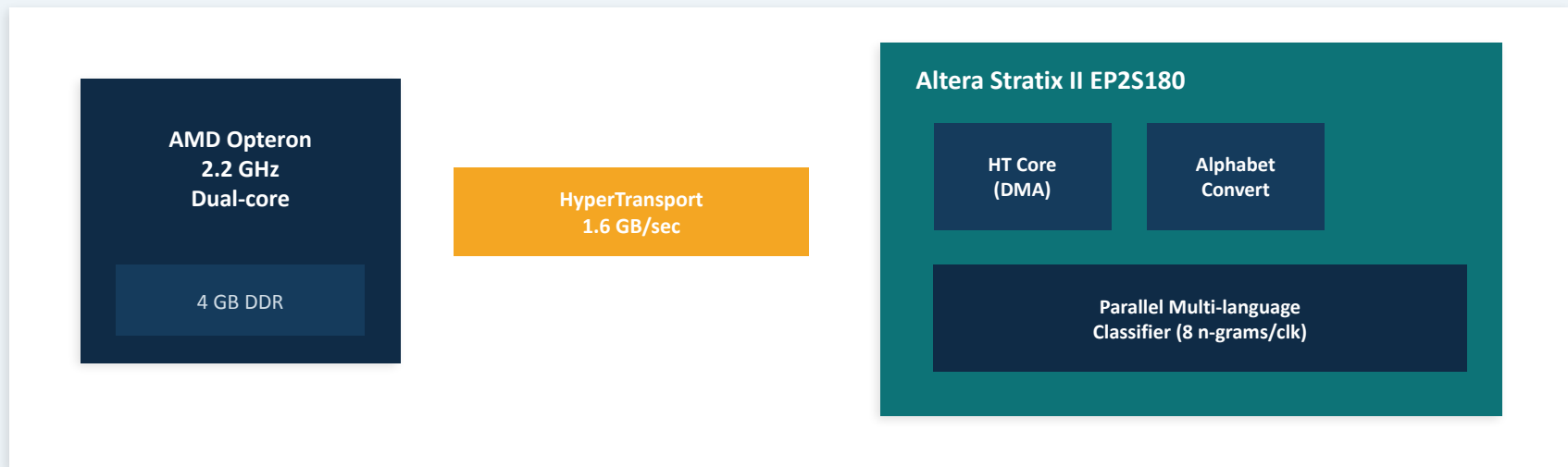


Each Bloom Filter tests two input n-grams per clock (dual-ported RAMs). All p language classifiers operate in parallel.

Scalability via Replication

The entire multiple language classifier can be replicated to process multiple document n-grams simultaneously. With 4 copies, the system tests 8 n-grams per clock from the input stream. An adder tree aggregates match counts after processing each document. This is where the on-chip Bloom Filter design provides its key advantage over HAIL's off-chip SRAM approach.

XtremeData XD1000 System Architecture



Processor

AMD Opteron
2.2 GHz dual-core

FPGA

Altera Stratix II
EP2S180F1508-C3

Interconnect

HyperTransport
1.6 GB/sec peak

On-chip RAM

768 x 4 Kbit
embedded RAMs

Implementation Details



Alphabet Conversion

8-bit extended ASCII $\rightarrow$ 5-bit code. Uppercase conversion, accent mapping. Reduces n-gram size from 32 to 20 bits, saving RAM address space.



N-gram Generation

Sliding window buffers characters and generates a 4-gram at each position. Treats input as continuous character stream, oblivious to word boundaries.



Data Transfer (DMA)

Host CPU sends documents via DMA from DDR memory. Size command precedes each document. Watchdog timer handles errors. End-of-Document delimiter between files.



Result Retrieval

Query Result command returns match counts per language. XOR checksum and status bits verify correct transfer. Software identifies language with highest count.



VHDL Implementation

Coded in VHDL, compiled with Altera Quartus II v7.1. Portable design — can be compiled for Xilinx FPGAs using ISE tool chain.



Async vs. Sync

Asynchronous DMA (no interrupts) doubles throughput compared to synchronous version: 470 vs. 228 MB/sec. Critical for streaming architecture.

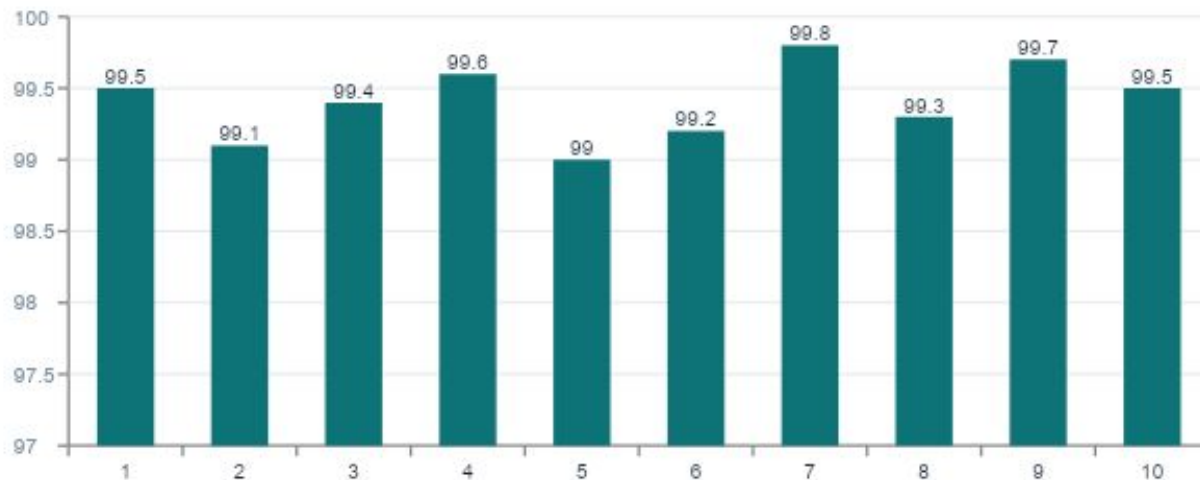
Classification Accuracy

Test Dataset

JRC-ACQUIS Multilingual Parallel Corpus v3.0 — EU law across 22 European languages. 10 languages tested: Czech, Slovak, Danish, Swedish, Spanish, Portuguese, Finnish, Estonian, French, English. ~5,700 docs per language, avg 1,300 words each.

Configuration

k = 4 hash functions, m = 16 Kbits per bit-vector, t = 5,000 n-grams per profile (n = 4). 10% corpus for training, 90% for testing. Expected false positive rate: 5 per thousand.



99.45%

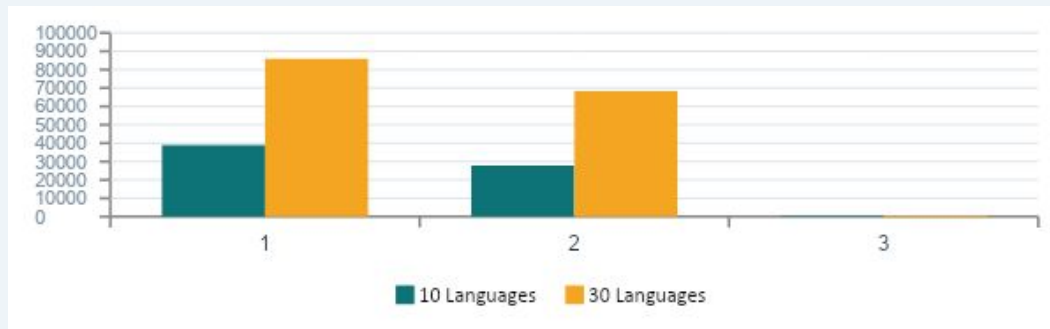
Average Accuracy
across 10 languages
(range: 99.05%–99.76%)

FPGA Resource Utilization

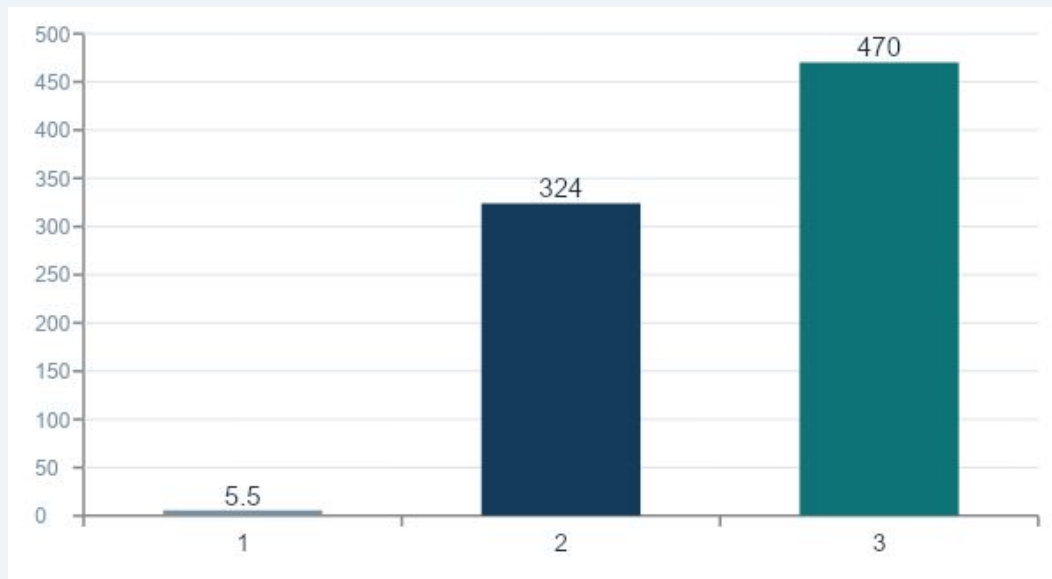
Metric	10 Languages (k=4, m=16Kb)	30 Languages (k=6, m=4Kb)
Logic Utilization	38,891	85,924
Logic Registers	27,889	68,423
M4K Blocks	680	768
M-RAMs	9	6
Clock Frequency	194 MHz	170 MHz
N-grams / Clock	8	8

Analysis

The limiting factor is on-chip embedded RAM. Using smaller bit-vectors (4 Kbit) with more hash functions (k=6) is the most space-efficient config, supporting 3× more languages with only a marginal accuracy loss.



Throughput & Performance Comparison



85×

vs. Software

Mguesser on AMD Opteron
2.4 GHz, 16 GB RAM

1.45×

vs. HAIL FPGA

Xilinx XCV2000E-8
off-chip SRAM based

1.4
GB/s
Theoretical Peak

194 MHz × 8 n-grams/clock
= 1,552M n-grams/sec

Synchronous vs. Asynchronous DMA

Switching from interrupt-based synchronization to asynchronous DMA doubled throughput (228 → 470 MB/sec). The current bottleneck is the XtremeData system bandwidth (500 MB/sec), not the classifier itself. Performance will scale with improved communication infrastructure.

Conclusions & Key Takeaways



Bloom Filters for Classification

Replacing direct lookup tables with Parallel Bloom Filters enables on-chip storage, eliminating the off-chip SRAM bottleneck of HAIL.



Scalable Design

Designers can trade off between k , m , and accuracy to support 10–30 languages on a single FPGA with >99% accuracy.



85× Faster than Software

470 MB/sec achieved; theoretical peak of 1.4 GB/sec. Hardware parallelism at multiple levels: hash functions, languages, documents.



Portable & Flexible

No dependence on off-chip memory. Easily portable between Altera and Xilinx FPGAs. Design scales with improved communication bandwidth.